


UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ingeniería

Compilers
Parser (LL(1)) and Syntax-Directed Translation (SDT)

Students:

Team: 02

Cortés Guillén Pavel Francisco – 118001568
Cojulum Ortega Mabel Ximena – 321101921
Gómez Meza Jorge Ángel – 118003225
Monroy Tapia Jesus Alejandro – 321045979
Villa Garcia Arely Guadalupe – 321000697

Professor: **René Adrián Dávila Pérez**
Group: 5 *Semester:* 2027-1

Deadline: November 10th, 2026

Contents

1 Introduction **3**
1.1 Objectives . . . 3
1.2 Scope . . . 3
1.3 Implementation approach . . . 3
1.4 Organisation of this report . . . 3

2 Theoretical Framework **4**
2.1 Syntax analysis and context-free grammars . . . 4
2.2 Top-down parsing and LL(1) grammars . . . 4
2.3 Grammar transformations . . . 4
2.4 FIRST and FOLLOW sets . . . 5
2.5 LL(1) parsing table and the predictive algorithm . . . 5
2.6 Syntax-Directed Translation . . . 5
2.7 Symbol table and type checking . . . 5

3 Development **6**
3.1 Selection of the parsing method . . . 6
3.2 Architecture and integration . . . 6
3.3 Input: tokens from the Lexer . . . 7
3.4 Context-free grammar . . . 7
3.5 FIRST and FOLLOW sets . . . 8
3.6 LL(1) parsing table . . . 8
3.7 Predictive parser implementation . . . 9
3.8 Parse tree . . . 9
3.9 Symbol table . . . 9
3.10 Semantic analysis: SDT scheme . . . 10
3.11 Control of outcomes . . . 11
3.12 Web interface . . . 11

4 Results **11**
4.1 Theoretical justification of each output . . . 11
4.1.1 Output 1: `Parsing Success!` and `SDT Verified!` . . . 11
4.1.2 Output 2: `Parsing Success!` and `SDT error...` . . . 12
4.1.3 Output 3: `Parsing error...` . . . 12
4.2 Symbol table results . . . 13
4.3 Manual test cases . . . 13
4.4 Regression after corrections . . . 13
4.5 Automated testing . . . 14
4.6 Interface evidence . . . 16

4.7 Version control and delivery status 18

5 Conclusions **18**

1 Introduction

Compilation is organised as a pipeline of phases, each one consuming the output of the previous one. In the first deliverable of this project we built a hand-crafted lexical analyzer (Lexer) based on a 13-state deterministic finite automaton, able to recognise 40 token types and to recover from lexical errors through Panic Mode [8]. That component answers the question *"do the lexemes belong to the language?"*. It cannot answer whether the sequence of tokens forms valid statements, nor whether those statements obey basic meaning rules such as "a variable must be declared before it is used".

This second deliverable extends the compiler of Team 02 with two new phases:

- a **syntax analyzer (parser)**, which validates the token stream against a context-free grammar (CFG) and builds a parse tree; and
- a **semantic analyzer based on Syntax-Directed Translation (SDT)**, which traverses the tree to check declarations, uses, scopes and data types, using the symbol table shared with the Lexer.

1.1 Objectives

1. Receive as input the tokens produced by the Lexer (the parser never re-tokenises the source text).
2. Define a grammar whose productions match the token categories of the language.
3. Choose a parsing method, build the parse tree and define the SDT rules for semantic analysis.
4. Read the source from a string or a file and report the line of every error.
5. Distinguish the three possible outcomes: *parsing error*, *parsing success with SDT error*, and *successful compilation*, justifying each one theoretically.

1.2 Scope

The implementation covers lexical, syntactic and semantic analysis, the symbol table update, the integration of the full flow and a web interface. It does *not* generate machine code, execute the source program or optimise code.

1.3 Implementation approach

The assignment offers two options: applying only the theoretical concepts seen in class (which grants extra points), or using tools and libraries that hide the process (a "black box") [9]. Our team chose the first option. The parser and the SDT were written by hand in Python, without parser generators such as PLY, SLY, YACC or Bison. The parser uses an explicit stack, an input buffer and an LL(1) table, so that every step of the algorithm can be traced and related to the theory.

1.4 Organisation of this report

Section 2 presents the theoretical framework. Section 3 describes the design and the implementation, applying each concept to our language. Section 4 presents the results, with the theoretical argument for each output. Section 5 closes with conclusions and future work.

2 Theoretical Framework

2.1 Syntax analysis and context-free grammars

The syntax analyzer takes the stream of tokens produced by the lexical analyzer and verifies that it can be generated by the grammar of the language; as a by-product it produces a tree that represents the syntactic structure of the program [1]. The syntax of programming languages is specified with *context-free grammars*, a class of formal grammars introduced by Chomsky [3].

A CFG is a 4-tuple $G = (N, \Sigma, P, S)$ where N is the set of non-terminals, Σ the set of terminals (here, the token categories emitted by the Lexer), P a set of productions $A \to \alpha$ with $A \in N$ and $\alpha \in (N \cup \Sigma)^*$, and $S \in N$ the start symbol. A *derivation* rewrites non-terminals step by step; in a *leftmost derivation* the leftmost non-terminal is always the one rewritten. A *parse tree* shows a derivation graphically: the root is S, each internal node is a non-terminal whose children are the symbols on the right-hand side of the applied production, and the leaves, read from left to right, form the input string.

2.2 Top-down parsing and LL(1) grammars

Parsing methods are classified as top-down (the tree is built from the root towards the leaves) and bottom-up (from the leaves towards the root; LR(0), SLR(1), CLR(1), LALR(1)) [1, 2]. A *predictive* top-down parser chooses the production to apply by looking only at the next input symbol (*lookahead*), so it needs no backtracking. The class of grammars that can be parsed this way with $k = 1$ symbol of lookahead is called LL(1): the first **L** stands for scanning the input from *left* to right, the second **L** for producing a *leftmost* derivation, and the **1** for one symbol of lookahead [4, 6, 5].

A grammar is LL(1) if, for any two distinct productions $A \to \alpha \mid \beta$:

1. $\mathrm{FIRST}(\alpha) \cap \mathrm{FIRST}(\beta) = \emptyset$;
2. at most one of α, β derives ε; and
3. if $\beta \Rightarrow^* \varepsilon$, then $\mathrm{FIRST}(\alpha) \cap \mathrm{FOLLOW}(A) = \emptyset$.

LL(1) grammars cannot be left-recursive, ambiguous or contain common prefixes among the alternatives of a non-terminal.

2.3 Grammar transformations

Elimination of left recursion. A production $A \to A\alpha \mid \beta$ makes a top-down parser loop forever, because expanding A calls A again without consuming input. It is replaced by the equivalent pair

$$A \to \beta A', \qquad A' \to \alpha A' \mid \varepsilon.$$

We use this transformation to write the arithmetic, relational and logical expression levels.

Left factoring. When two alternatives share a prefix, $A \to \alpha\beta_1 \mid \alpha\beta_2$, one lookahead symbol cannot decide between them. The common prefix is extracted:

$$A \to \alpha A', \qquad A' \to \beta_1 \mid \beta_2.$$

This is the transformation already used in the first deliverable [8] for the productions beginning with `int` and `printf`, and it is applied here to *DeclStmt*, *DeclRest* and *ArgList*.

2.4 FIRST and FOLLOW sets

FIRST(α) is the set of terminals that can begin a string derived from α (plus ε if $\alpha \Rightarrow^* \varepsilon$). FOLLOW($A$) is the set of terminals that can appear immediately to the right of A in some sentential form (with \$ marking the end of input). They are computed with the following rules [1]:

- If X is a terminal, FIRST(X) = $\{X\}$.
- If $X \to \varepsilon$, add ε to FIRST(X).
- If $X \to Y_1 Y_2 \dots Y_k$, add FIRST(Y_1) $\setminus \{\varepsilon\}$ to FIRST(X); if $Y_1 \Rightarrow^* \varepsilon$ continue with Y_2, and so on; add ε if all Y_i derive ε.
- \$ $\in$ FOLLOW(S).
- For $A \to \alpha B \beta$, add FIRST(β) $\setminus \{\varepsilon\}$ to FOLLOW(B).
- For $A \to \alpha B$, or $A \to \alpha B \beta$ with $\varepsilon \in$ FIRST(β), add FOLLOW(A) to FOLLOW(B).

2.5 LL(1) parsing table and the predictive algorithm

The table $M[A, a]$ is filled as follows: for each production $A \to \alpha$, (i) put it in $M[A, a]$ for every terminal $a \in$ FIRST(α); (ii) if $\varepsilon \in$ FIRST(α), put it in $M[A, b]$ for every $b \in$ FOLLOW(A). A cell with two productions is a conflict (the grammar is not LL(1)); an empty cell denotes a syntax error.

The table-driven parser keeps a stack (initially \$ followed by the start symbol) and an input buffer ending in \$. Let X be the top of the stack and a the lookahead:

1. if $X = a =$ \$: **accept**;
2. if X is a terminal and $X = a$: pop X and advance the input (*match*);
3. if X is a terminal and $X \neq a$: **error**;
4. if X is a non-terminal and $M[X, a] = X \to Y_1 \dots Y_k$: pop X and push $Y_k, \dots, Y_1$ (so that Y_1 is on top);
5. if $M[X, a]$ is empty: **error**.

Time and space are linear in the size of the input.

2.6 Syntax-Directed Translation

A syntax-directed definition attaches *semantic rules* to the productions of a CFG, computing *attributes* of the grammar symbols. An attribute is *synthesized* if its value at a node depends only on the attributes of its children; it is *inherited* if it depends on its parent or siblings [7, 1]. A *syntax-directed translation scheme* embeds the semantic actions in the productions and fixes the order in which they run. A common implementation, and the one used here, is a two-step procedure: (1) the parser validates the syntax and builds the parse tree; (2) the tree is traversed in postorder (bottom-up evaluation of synthesized attributes), executing the semantic actions. Separating the two steps cleanly distinguishes a syntax failure from a semantic failure.

2.7 Symbol table and type checking

The *symbol table* stores, for every identifier, the attributes needed by later phases (type, size, scope, line of declaration and use, memory address) [1, 2]. The usual operations are `Insert`, `Lookup`, `Set` and `Reset`; the latter restores the table when a block ends, implementing block scope. *Type checking* verifies that each operator is applied to operands of compatible types and that the type of an expression matches the type required by its context (assignment, condition, argument). Our language uses a *strict* type system with no implicit conversions.

3 Development

3.1 Selection of the parsing method

We selected a **predictive top-down LL(1) parser, without backtracking, implemented with an explicit stack**. The decision rests on three criteria:

1. **Traceability for the report and the oral defence.** Recursive descent hides the derivation state in Python's call stack. With an explicit stack we can print, at every iteration, the quadruple (*Step, Stack, Input, Action*), which gives exactly the theoretical argument required for each output.
2. **Advantage over bottom-up methods.** LR-family parsers need the canonical collection of items (closure and goto) and may show shift/reduce conflicts as the language grows. The LL(1) model works directly on FIRST and FOLLOW sets with linear time and space.
3. **Separation of syntax and semantics.** The two-step SDT procedure (parse tree first, semantic traversal second) lets us separate *Parsing Success!* followed by *SDT error...* from a syntactic failure.

3.2 Architecture and integration

The existing package `unam.fi.compilers.g5.02` was extended, preserving the Lexer and adding the new layers. The main module orchestrates the flow and a symbol table is shared by the Lexer and the SDT (Figure 1).

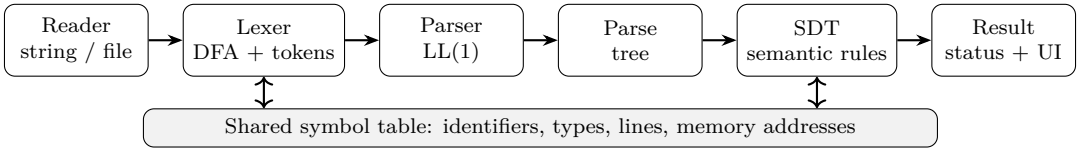


Figure 1: Logical flow from reading the source to presenting the results.

Module	Responsibility
`main.py`	Orchestrates Reader → Scanner → Parser → SDT. Keeps the lexical-only functions and adds `run_compiler()` and `run_compiler_from_file()`.
`io/reader.py`, `io/writer.py`	Reading of strings/files and report generation.
`lexer/dfa.py`, `lexer/scanner.py`	Manual lexeme recognition, positions and errors (reused from deliverable 1).
`parser/grammar.py`	Productions, FIRST/FOLLOW computation and predictive table.
`parser/ll1_parser.py`	LL(1) stack, input buffer, terminal matching, tree construction and trace.
`models/ast_node.py`	Tree node: symbol, token, children, synthesized type and line.
`sdt/analyzer.py`	Tree traversal and semantic checks.
`symbol_table/symtab.py`	Identifiers and attributes: type, size, dimension, lines, address, declaration state.
`web/app.py`, templates, static	Web interface that presents the compilation results.

Table 1: Module organisation.

Execution flow. (1) The Reader obtains the source. (2) The Scanner forms the lexemes, generates tokens and records lexical errors; identifiers are inserted early in the symbol table with pending attributes. (3) The parser receives the tokens, appends the end-of-input marker $, consults the table and builds the parse tree. (4) Only if the parser finishes successfully, the SDT traverses the tree validating declarations, uses, types and statement rules. (5) `main.py` gathers state, messages, tokens, tree, trace and symbol table, which the web application displays.

3.3 Input: tokens from the Lexer

The parser consumes the token list of the Lexer, whose 40 internal types are grouped for presentation into five categories (keyword, identifier, operator, constant, punctuation) [8]. For the lookahead decision the parser uses the token type or, for keywords, operators and punctuators, the specific lexeme (`int`, `if`, `;`, `==`, ...). The terminal **id** denotes an identifier token, **const** denotes `INTEGER` or `REAL_NUMBER`, and **lit** denotes `BOOL_LITERAL` (`True`, `False`) or `STRING_LITERAL`.

The end-of-input marker $ is a synthetic token appended to the buffer; it is not part of the catalogue of 40 lexical tokens. The grammar was adapted to the real Lexer: identifiers are plain names such as `x`, and the logical operators are the words `and`, `or` and `not`.

3.4 Context-free grammar

Toward a deterministic grammar, we applied left-recursion elimination (expression levels) and left factoring (declarations, calls and optional parts). Statement bodies of control structures are delimited by braces, `{ StmtList }`, which avoids the classic dangling-else ambiguity, and every simple statement requires a terminating semicolon. The resulting grammar has 24 rules (Table 2); expression levels encode operator precedence, from lowest to highest: logical, relational/equality, additive, multiplicative, unary `not` and primary factors.

Table 2: Final CFG (24 rules). Terminals in `typewriter`.

#	Production
	A. Program structure and statements
1	*Program* → *StmtList*
2	*StmtList* → *Stmt StmtList* | ε
3	*Stmt* → *DeclStmt* | *AssignStmt* | *PrintStmt* | *IfStmt* | *WhileStmt* | *ForStmt*
4	*DeclStmt* → *Type* `id` *DeclRest* `;`
5	*DeclRest* → `=` *E* | ε
6	*AssignStmt* → `id =` *E* `;`
7	*PrintStmt* → `printf (` *E ArgList* `) ;`
8	*ArgList* → `,` *E ArgList* | ε
9	*IfStmt* → `if (` *E* `) {` *StmtList* `}` *ElsePart*
10	*ElsePart* → `else {` *StmtList* `}` | ε
11	*WhileStmt* → `while (` *E* `) {` *StmtList* `}`
12	*ForStmt* → `for (` *ForInit* `;` *E* `;` *ForUpdate* `) {` *StmtList* `}`
13	*ForInit* → *Type* `id =` *E* | `id =` *E*
14	*ForUpdate* → `id =` *E*
15	*Type* → `int` | `real` | `string` | `bool`
	B. Expressions without left recursion

#	Production
16	$E \to R\ E'$
17	$E' \to \mathit{OpLog}\ R\ E' \mid \varepsilon, \quad \mathit{OpLog} \in \{\texttt{and}, \texttt{or}\}$
18	$R \to A\ R'$
19	$R' \to \mathit{OpRel}\ A\ R' \mid \varepsilon, \quad \mathit{OpRel} \in \{\texttt{<}, \texttt{>}, \texttt{<=}, \texttt{>=}, \texttt{==}, \texttt{!=}\}$
20	$A \to T\ A'$
21	$A' \to \mathit{OpAdit}\ T\ A' \mid \varepsilon, \quad \mathit{OpAdit} \in \{\texttt{+}, \texttt{-}\}$
22	$T \to F\ T'$
23	$T' \to \mathit{OpMult}\ F\ T' \mid \varepsilon, \quad \mathit{OpMult} \in \{\texttt{*}, \texttt{/}\}$
24	$F \to \texttt{(}\ E\ \texttt{)} \mid \texttt{not}\ F \mid \texttt{id} \mid \texttt{const} \mid \texttt{lit}$

3.5 FIRST and FOLLOW sets

Applying the rules of Section 2.4 to the grammar of Table 2 gives the sets of Table 3. Let $\mathcal{T} = \{\texttt{int}, \texttt{real}, \texttt{string}, \texttt{bool}\}$ and $\mathrm{F}_S = \mathrm{FIRST}(\mathit{Stmt}) = \mathcal{T} \cup \{\texttt{id}, \texttt{printf}, \texttt{if}, \texttt{while}, \texttt{for}\}$; let $\mathrm{F}_E = \{\texttt{(}, \texttt{not}, \texttt{id}, \texttt{const}, \texttt{lit}\}$.

Non-terminal	FIRST	FOLLOW
Program	$\mathrm{F}_S \cup \{\varepsilon\}$	$\{\$\}$
StmtList	$\mathrm{F}_S \cup \{\varepsilon\}$	$\{\texttt{}\}, \$\}$
Stmt, *DeclStmt*, *AssignStmt*, *PrintStmt*, *IfStmt*, *WhileStmt*, *ForStmt*	each its own initial terminals (*Stmt*: F_S)	$\mathrm{F}_S \cup \{\texttt{}\}, \$\}$
DeclRest	$\{\texttt{=}, \varepsilon\}$	$\{\texttt{;}\}$
ArgList	$\{\texttt{,}, \varepsilon\}$	$\{\texttt{)}\}$
ElsePart	$\{\texttt{else}, \varepsilon\}$	$\mathrm{F}_S \cup \{\texttt{}\}, \$\}$
ForInit	$\mathcal{T} \cup \{\texttt{id}\}$	$\{\texttt{;}\}$
ForUpdate	$\{\texttt{id}\}$	$\{\texttt{)}\}$
Type	$\mathcal{T}$	$\{\texttt{id}\}$
E, R, A, T, F	F_E	see below
E'	$\{\texttt{and}, \texttt{or}, \varepsilon\}$	$\{\texttt{;}, \texttt{)}, \texttt{,}\}$
R'	$\{\texttt{<}, \texttt{>}, \texttt{<=}, \texttt{>=}, \texttt{==}, \texttt{!=}, \varepsilon\}$	$\{\texttt{and}, \texttt{or}, \texttt{;}, \texttt{)}, \texttt{,}\}$
A'	$\{\texttt{+}, \texttt{-}, \varepsilon\}$	$\mathrm{FOLLOW}(R') \cup \{\mathrm{OpRel}\}$
T'	$\{\texttt{*}, \texttt{/}, \varepsilon\}$	$\mathrm{FOLLOW}(A') \cup \{\texttt{+}, \texttt{-}\}$

Table 3: FIRST and FOLLOW sets. For expressions: $\mathrm{FOLLOW}(E) = \mathrm{FOLLOW}(E') = \{\texttt{;}, \texttt{)}, \texttt{,}\}$; $\mathrm{FOLLOW}(R)$ adds $\{\texttt{and}, \texttt{or}\}$; $\mathrm{FOLLOW}(A)$ adds the relational operators; $\mathrm{FOLLOW}(T)$ adds $\{\texttt{+}, \texttt{-}\}$; $\mathrm{FOLLOW}(F)$ adds $\{\texttt{*}, \texttt{/}\}$.

3.6 LL(1) parsing table

Following the construction rules of Section 2.5, Table 4 lists the production selected for each lookahead. No terminal appears in two cells of the same row: the grammar has no conflicts and is strictly LL(1). Remarkably, `else` is not in $\mathrm{FOLLOW}(\mathit{Stmt})$, so $\mathit{ElsePart} \to \varepsilon$ never competes with $\mathit{ElsePart} \to \texttt{else}\ldots$. Any combination not listed is an empty cell, i.e. a syntax error. The

table is generated automatically by `parser/grammar.py`.

Non-terminal	Production selected	Lookahead terminals
Program	→ *StmtList*	$F_S \cup \{\$\}$
StmtList	→ *Stmt StmtList*	F_S
	→ ε	`}`, $
Stmt	→ *DeclStmt* / *AssignStmt* / *PrintStmt* / *IfStmt* / *WhileStmt* / *ForStmt*	$\mathcal{T}$ / `id` / `printf` / `if` / `while` / `for`
DeclRest	→ `=` *E* | → ε	`=` | `;`
ArgList	→ `,` *E ArgList* | → ε	`,` | `)`
ElsePart	→ `else {`*StmtList*`}` | → ε	`else` | $F_S \cup \{\}, \$\}$
ForInit	→ *Type* `id =` *E* | → `id =` *E*	$\mathcal{T}$ | `id`
Type	→ `int` / `real` / `string` / `bool`	the corresponding keyword
E, *R*, *A*, *T*	→ *R E′*, *A R′*, *T A′*, *F T′*	F_E
E′	→ *OpLog R E′* | → ε	`and`, `or` | `;`, `)`, `,`
R′	→ *OpRel A R′* | → ε	relational ops. | `and`, `or`, `;`, `)`, `,`
A′	→ *OpAdit T A′* | → ε	`+`, `-` | FOLLOW(A')
T′	→ *OpMult F T′* | → ε	`*`, `/` | FOLLOW(T')
F	→ `(` *E* `)` / `not` *F* / `id` / `const` / `lit`	`(` / `not` / `id` / `const` / `lit`

Table 4: Compact representation of the LL(1) parsing table $M[A, a]$ (*IfStmt*, *WhileStmt*, *ForStmt*, *DeclStmt*, *AssignStmt* and *PrintStmt* have a single production, selected on their first terminal).

3.7 Predictive parser implementation

`parser/ll1_parser.py` implements the algorithm of Section 2.5. Initially the stack holds the end marker $ and the start symbol *Program*, and a synthetic token with lexeme $ is appended to the input buffer. At each iteration the top of the stack is compared with the lookahead: matching terminals consume input; for a non-terminal, the production in $M[X, a]$ is selected, its child nodes are created and pushed in reverse order so that they are processed from left to right. ε-productions are applied through the FOLLOW sets.

The parser records a **trace** of four fields per step (*Step, Stack, Input, Action*), including expansions, matches, ε applications and the exact point where an empty cell or a terminal mismatch is detected. This trace is the evidence of the algorithm, not only a success/error message.

3.8 Parse tree

Each tree node (`ASTNode`) keeps the grammar symbol, the token when the node is a terminal, the ordered list of children, the associated line, and the `data_type` attribute synthesized later by the SDT. The tree preserves the structure of the derivation and is the only input of the semantic analyzer.

3.9 Symbol table

The table is shared by the Scanner and the SDT. The Lexer inserts every identifier it finds, but initially does not know whether it belongs to a valid declaration; the SDT completes the attributes when it confirms a declaration or detects a use.

Attribute	Meaning
`name`	Identifier name (key).
`type`	`int`, `real`, `bool` or `string`; starts as `TBD`.
`size`	Size in bytes; starts as `TBD`.
`dimension`	0 for scalar variables.
`first_appearance_line`	First line where the Lexer found the identifier.
`declaration_line`	Line of the declaration confirmed by the SDT; starts as `None`.
`line_usage`	List of lines where the identifier is used.
`address`	Address assigned after a declaration is confirmed.

Table 5: Symbol table entry attributes.

Sizes are: `int` 4 bytes, `real` 8, `bool` 1 and `string` 16. The base address is `0x1000`; when a declaration is confirmed, the entry receives the current address and the next free address advances by the size of the type (e.g. `0x1000`, then `0x1004` after an `int`).

Design correction. Testing revealed that the first version stored the first line where an identifier appeared in `declaration_line`. An identifier only used in an assignment, or whose declaration had been rejected, could therefore look as if it had been declared. The two data were separated: `first_appearance_line` is a lexical fact, while `declaration_line` is only set when the SDT confirms the declaration. This also keeps a single entry per identifier and prevents a duplicate declaration from overwriting the first valid one.

3.10 Semantic analysis: SDT scheme

Semantic analysis runs only if the parser accepted the whole input. The analyzer traverses the parse tree in postorder, synthesizing the type of expressions and applying the rules of Table 6. Any violation synthesizes the attribute `error_type` and triggers `SDT error...` with the line.

Production	Semantic rule
DeclStmt → *Type* `id` *DeclRest* `;`	`Lookup(id)`: if already declared, error "redeclared variable". If there is an initialiser, its type must equal `Type`; otherwise error. Only then `Set(id, type, size, dimension=0)` and the address are confirmed.
AssignStmt → `id =` *E* `;`	`Lookup(id)`: if type is `TBD` (or the variable is absent), error "undeclared variable"; if `id.type` ≠ `E.type`, error "type mismatch in assignment"; else record the line of use.
PrintStmt → `printf (` *E ArgList* `) ;`	Evaluate the main expression and every extra argument; if any yields `error_type`, error "invalid printf argument".
IfStmt, *WhileStmt*, *ForStmt*	The condition *E* must have type `bool`; otherwise error. The bodies (and *ForInit*, *ForUpdate*, *ElsePart*) are verified recursively.
E, *R*, *A*, *T* and primed forms	Combine the operand types following Table 7; an invalid combination yields `error_type`.
F → `(`*E*`)`	`F.type` = `E.type`.
F → `not` *F*	Requires `bool`; result `bool`.
F → `const` | `lit` | `id`	`const`: `int` or `real` (if it contains a point); `lit`: `bool` or `string`; `id`: `Lookup(id).type` (error if `TBD`) and record the line of use.
Blocks `{ ... }`	Variables declared inside a block are removed (`Reset`) when the block ends; outer-scope variables remain visible.

Table 6: SDT rules for statements and expressions.

Type compatibility. The language is strictly typed over four primitive types; no automatic conversion between `int` and `real` is applied (Table 7). Parenthesised expressions keep the type of the inner expression, and relational operators over numeric values produce `bool`.

Type	Arithmetic (+ - * /)	Relational (< > <= >=)	Equality (== !=)	Logical (and or not)	Assignment
`int`	int → int	int → bool	int → bool	forbidden	only `int`
`real`	real → real	real → bool	real → bool	forbidden	only `real`
`string`	only +: string → string	forbidden	string → bool	forbidden	only `string`
`bool`	forbidden	forbidden	bool → bool	bool → bool	only `bool`

Table 7: Semantic type compatibility matrix.

3.11 Control of outcomes

The integrated flow (`run_compiler(source)`) distinguishes four states (Table 8).

State	Behaviour
Lexical error	The Scanner records errors; the integrated flow stops before running the parser.
Syntax error	The parser rejects the input and reports the line (`Parsing error...`, `error in line n`). The SDT is not executed, so no semantic message is shown.
Semantic error	The parser accepts (`Parsing Success!`) but the SDT reports `SDT error...` and the line.
Successful compilation	Parser and SDT finish: `Parsing Success!` and `SDT Verified!`.

Table 8: Outcome states of the integrated compiler.

3.12 Web interface

The web application (Flask) shows the editor with the source code, a summary (tokens, errors, lexical categories, supported types, overall status), the token table (type, category, lexeme, line, column), the parse tree, the LL(1) trace, the SDT result and the symbol table with first-appearance and declaration columns (a dash when no declaration was confirmed), plus the catalogue of the 40 token types. It is run as a module to keep relative imports working:

```
cd <project>/Lexer
python -m develop.web.app # then open http://127.0.0.1:5000
```

4 Results

4.1 Theoretical justification of each output

4.1.1 Output 1: `Parsing Success!` and `SDT Verified!`

Input: `int x = 10;` **Tokens:** `KW(int)`, `ID(x)`, `ASSIGN`, `INTEGER(10)`, `SEMICOLON`, $.

Table 9 shows the condensed trace. The parser reaches the configuration (stack = $, input = $), which by the algorithm of Section 2.5 guarantees that the token string belongs to $L(G)$.

The SDT then traverses the tree in postorder: the constant `10` synthesizes `int`, the type propagates up to E through F, T, A, R; in *DeclStmt*, `Lookup(x)` confirms the entry is still `TBD` (not redeclared) and `Type=int=E.type`, so `Set(x, int, 4, 0)` assigns address `0x1000`.

Step	Stack (base → top)	Input	Action
1	\$ *Program*	`int x = 10 ; $`	*Program* → *StmtList*
2	\$ *StmtList*	`int x = 10 ; $`	*StmtList* → *Stmt StmtList*
3	\$ *StmtList Stmt*	`int x = 10 ; $`	*Stmt* → *DeclStmt*
4	\$ *StmtList DeclStmt*	`int x = 10 ; $`	*DeclStmt* → *Type* `id` *DeclRest* `;`
5	\$ *StmtList* ; *DeclRest* id *Type*	`int x = 10 ; $`	*Type* → `int`, then match `int`
6	\$ *StmtList* ; *DeclRest* id	`x = 10 ; $`	match `id`
7	\$ *StmtList* ; *DeclRest*	`= 10 ; $`	*DeclRest* → `=` *E*, then match `=`
8	\$ *StmtList* ; *E*	`10 ; $`	$E \Rightarrow R \Rightarrow A \Rightarrow T \Rightarrow F$; $F \rightarrow$ `const` (via $M[\cdot, \texttt{const}]$)
9	\$ *StmtList* ; E' R' A' T' const	`10 ; $`	match `const`
10	\$ *StmtList* ; E' R' A' T'	`; $`	ε-rules for T', A', R', E' (`;` ∈ FOLLOW)
11	\$ *StmtList* ;	`; $`	match `;`
12	\$ *StmtList*	\$	*StmtList* → ε
13	\$	\$	**Parsing Success!**

Table 9: Condensed LL(1) trace for `int x = 10;`.

```
Parsing Success!
SDT Verified!
```

4.1.2 Output 2: `Parsing Success!` and `SDT error...`

Inputs: `int x = "diez";  x = 5;  int x = 10; x = "hola";  if (10 + 20) { x = 1; }`.

All of them satisfy 100% of the syntactic rules of the CFG: in the first one, `"diez"` is a valid F (`lit`); in the second, an assignment with an identifier is a valid *AssignStmt*; in the last one, `10 + 20` is a valid E between parentheses followed by a braced body. Therefore the LL(1) parser empties the stack and prints `Parsing Success!`. The errors are context-dependent, so they cannot be expressed by a CFG; they are detected by the postorder traversal of the SDT: `int` ≠ `string` in the declaration; `x` still `TBD` in the assignment (undeclared); `int` ≠ `string` in the assignment; and the `IfStmt` rule requires `E.type = bool` but `10 + 20` synthesizes `int`.

```
Parsing Success!
SDT error...
error in line 1
```

4.1.3 Output 3: `Parsing error...`

Inputs: `int x = * 10;` `int x = 10` (missing semicolon).

For `int x = * 10;`, after matching `=` the stack top is E and the lookahead is `*`; since $* \notin \text{FIRST}(E)$ the cell $M[E, *]$ is empty (message *no production for M[E, *]*), which is a syntax error. For `int x = 10`, the top of the stack is the terminal `;` but the lookahead is \$, a terminal mismatch (`;` ≠ \$). In both cases the derivation cannot be completed, no valid tree exists and the SDT is **not** executed. The course instructions list *Parsing error...* together with *SDT error...* for the failing case [9]; our implementation prints only `Parsing error...` and the line, because semantic rules are attributes evaluated over a tree that does not exist when parsing fails (reporting a semantic error would be unjustified).

```
Parsing error...
error in line 1
```

4.2 Symbol table results

Input	Name	Type	Size	First app.	Declaration	Address
`int x = 10;`	x	int	4	1	1	`0x1000`
`x = 5;`	x	TBD	TBD	1	—	TBD
`int x = * 10;`	x	TBD	TBD	1	—	TBD
`bool activo = True;`	activo	bool	1	1	1	`0x1000`
`real precio = 12.5;`	precio	real	8	1	1	`0x1000`

Table 10: Symbol table contents after compiling each input independently.

4.3 Manual test cases

Input	Observed behaviour	Result
`int x = 10;`	Parser and SDT succeed; `x`: int, size 4, `0x1000`.	Correct
`x = 5;`	Parser succeeds; SDT reports undeclared variable.	Correct
`int x = "diez";`	Parser succeeds; SDT detects int/string incompatibility.	Correct
`int x = 10; int x = 20;`	SDT detects redeclaration on line 2; keeps first entry.	Correct
`int x = 10; x = "hola";`	SDT detects type mismatch in the assignment on line 2.	Correct
`bool activo = True;`	`bool` declaration accepted; size 1 byte.	Correct
`real precio = 12.5;`	`real` declaration accepted; size 8 bytes.	Correct

Table 11: Exploratory manual tests in the web interface.

Lexical and syntactic edge cases. `_variable123` is recognised as `ID` but, isolated, the parser rejects it because it does not complete a statement; `10.25` is a `REAL_NUMBER` and also an invalid isolated statement; `10.` is reported by the Lexer as a malformed number; `"Hola mundo` as an unclosed string; `@` as an illegal character with line and column; `""` is a valid empty `STRING_LITERAL`, rejected by the parser if isolated; and `int x = * 10;` yields an empty LL(1) cell, so the SDT is not executed.

4.4 Regression after corrections

Input	Verification
`int x = * 10;`	`Parsing error...` and the line; no `SDT error...`; `x`: first appearance 1, declaration —.
`x = 5;`	`Parsing Success!` followed by `SDT error...` (undeclared); first appearance 1; declaration —.
`int x = 10;`	`Parsing Success!` and `SDT Verified!`; first appearance 1; declaration 1; int; 4; `0x1000`.

Table 12: Manual regression tests after the last commit.

The last commit also removed `SDT error...` from the syntax-error output, separated `first_appearance_line` from `declaration_line`, added distinct columns in the interface, created three tests for the new metadata and updated two tests whose expectations matched the previous behaviour.

4.5 Automated testing

The suite (unit and integration tests) is executed from the `develop/` folder with the virtual environment active:

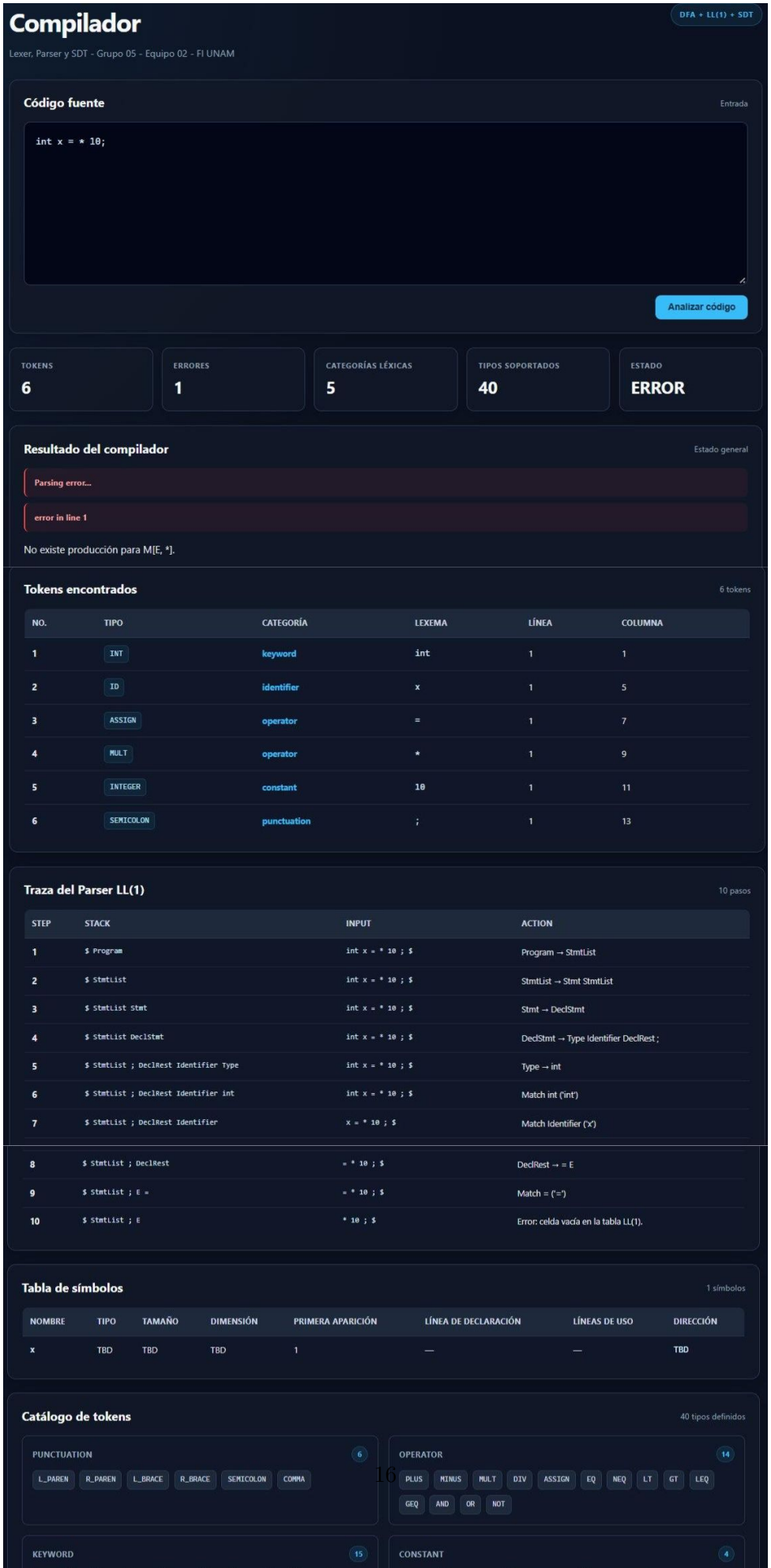
```
python -m unittest discover -s tests -t .. -v
```

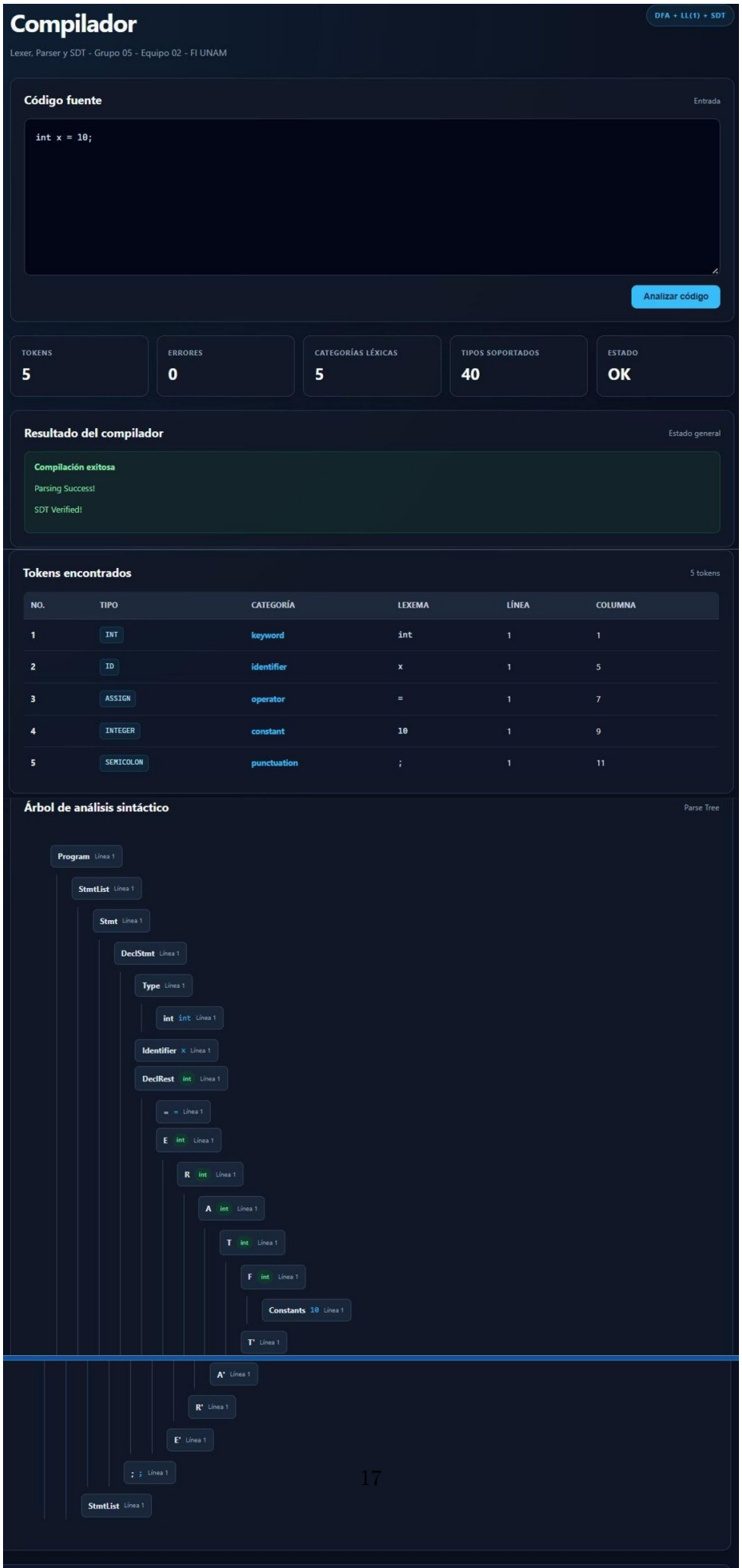
The option `-t ..` sets the parent folder as package root, required because tests use relative imports (e.g. `from ..lexer`).

Test group	**Run**	**Passed**
Parse tree (`test_ast_node.py`)	5	5
Compiler pipeline (`test_compiler_pipeline.py`)	5	5
Grammar (`test_grammar.py`)	4	4
LL(1) parser (`test_ll1_parser.py`)	11	11
Main program / CLI (`test_main.py`)	8	8
SDT analyzer (`test_sdt_analyzer.py`)	25	25
Declaration metadata (`test_symbol_declaration_metadata.py`)	3	3
Symbol table (`test_symtab_sdt.py`)	10	10
Web pipeline (`test_web_pipeline.py`)	6	6
Total	**77**	**77**

Table 13: Automated test results: 77 run, 77 passed, 0 failed, 0 errors.

4.6 Interface evidence




Compilador
DFA + LL(1) + SDT
Lexer, Parser y SDT - Grupo 05 - Equipo 02 - FI UNAM
Código fuente
Entrada
int x = 10;
Analizar código
TOKENS
5
ERRORES
0
CATEGORÍAS LÉXICAS
5
TIPOS SOPORTADOS
40
ESTADO
OK
Resultado del compilador
Estado general
Compilación exitosa
Parsing Success!
SDT Verified!
Tokens encontrados
5 tokens
NO. TIPO CATEGORÍA LEXEMA LÍNEA COLUMNA
1 INT keyword int 1 1
2 ID identifier x 1 5
3 ASSIGN operator = 1 7
4 INTEGER constant 10 1 9
5 SEMICOLON punctuation ; 1 11
Árbol de análisis sintáctico
Parse Tree
Program Línea 1
StmtList Línea 1
Stmt Línea 1
DeclStmt Línea 1
Type Línea 1
int int Línea 1
Identifier x Línea 1
DeclRest int Línea 1
= = Línea 1
E int Línea 1
R int Línea 1
A int Línea 1
T int Línea 1
F int Línea 1
Constants 10 Línea 1
T' Línea 1
A' Línea 1
R' Línea 1
E' Línea 1
; ; Línea 1
StmtList Línea 1

4.7 Version control and delivery status

The work was developed on the branch `feature/parser-sdt`. Pull Request #7 compares it against `main`; GitHub reports no conflicts. The relevant commits are `a204d30` (LL(1) parser foundation), `61ed95c` (symbol table extended for semantic analysis), `a9f3642` (SDT pipeline and web UI integration) and `93f7ae1` (parser error reporting and declaration metadata fix).

5 Conclusions

We extended the Lexer of Team 02 with a hand-written LL(1) parser and a semantic verifier based on SDT, using only the concepts seen in class. After eliminating left recursion and applying left factoring, the 24-rule grammar produced a conflict-free table, confirming that it is LL(1). The explicit stack and the trace turn every output into verifiable evidence of the algorithm.

The separation of phases was the central design decision. A string can fail while recognising tokens, while forming grammatical structures (context-free properties, detected in the table), or while obeying declaration and type rules (context-sensitive properties, detected by the SDT over the tree). This explains why a syntax error prevents semantic analysis, while a syntactically valid input can still be rejected by the SDT. Likewise, separating `first_appearance_line` from `declaration_line` was necessary to represent correctly the difference between a lexical finding and a semantic fact.

The validation, with 77 automated tests passed and manual regression cases, shows that the three required outcomes behave as justified in Section 4.

Limitations and future work. The compiler performs lexical, syntactic and semantic analysis only. Natural continuations are the generation of intermediate code (three-address code) from the validated tree, support for implicit conversions or arrays (dimension attribute), more informative syntax error messages with expected-token sets, and a recovery strategy for the parser (e.g. synchronising on FOLLOW sets) so that several syntax errors can be reported in one run.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[3] N. Chomsky, “On certain formal properties of grammars,” *Information and Control*, vol. 2, no. 2, pp. 137–167, 1959.

[4] P. M. Lewis II and R. E. Stearns, “Syntax-directed transduction,” *J. ACM*, vol. 15, no. 3, pp. 465–488, 1968.

[5] D. J. Rosenkrantz and R. E. Stearns, “Properties of deterministic top-down grammars,” *Information and Control*, vol. 17, no. 3, pp. 226–256, 1970.

[6] D. E. Knuth, “Top-down syntax analysis,” *Acta Informatica*, vol. 1, no. 2, pp. 79–110, 1971.

[7] D. E. Knuth, “Semantics of context-free languages,” *Math. Systems Theory*, vol. 2, no. 2, pp. 127–145, 1968.

[8] P. F. Cortés Guillén *et al.*, "Compilers: Lexer," Facultad de Ingeniería, UNAM, Mexico City, Mexico, Group 5, Team 02, Project Rep. 1, Oct. 2026.

[9] Facultad de Ingeniería, UNAM, "Parser & SDT: Syntax & Semantic Analyzer," Compilers course, Group 5, assignment instructions, Nov. 2026.